

Scripting Languages

Laszlo SZATHMARY
University of Debrecen
Faculty of Informatics

Lab #5

- set
- dictionary

(last update: 2026-08-25 [yyyy-mm-dd])



set

https://docs.python.org/3/library/stdtypes.html#set-types-set-frozenset

```
>>> basket = ['apple', 'ananas', 'banana', 'apple', 'orange', 'banana']
>>> fruits = set(basket)
>>> fruits
set(['orange', 'ananas', 'apple', 'banana'])
>>> type(fruits)
<type 'set'>
>>> li = list(fruits)
>>> li
['orange', 'ananas', 'apple', 'banana']
>>> type(li)
<type 'list'>
>>> sorted(li)
['ananas', 'apple', 'banana', 'orange']
>>> fruits
set(['orange', 'ananas', 'apple', 'banana'])
>>> 'kiwi' in fruits
False
>>> 'apple' in fruits
True
```

removing duplicates

Is an element in the set?

Exercise:

Consider the following elements: `[5, 2, 3, 5, 1, 4, -200, 5, 1, 3, 2, 2, 5]`.
Remove the duplicates, i.e. one element can be present in the result at most once. The elements in the result should be sorted.

(set01)

```
>>> a = ['apple', 'banana', 'lemon']
>>> a = set(a)
>>> a
set(['lemon', 'apple', 'banana'])
>>> b = set()
>>> b.add('banana')
>>> b.add('orange')
>>> b
set(['orange', 'banana'])
>>> a.union(b)
set(['orange', 'lemon', 'apple', 'banana'])
>>> a.intersection(b)
set(['banana'])
>>> a.difference(b)
set(['lemon', 'apple'])
>>> a
set(['lemon', 'apple', 'banana'])
>>> a.remove('lemon')
>>> a
set(['apple', 'banana'])
```

list to set

empty set;
extending a set

classic
set operations

removing an element

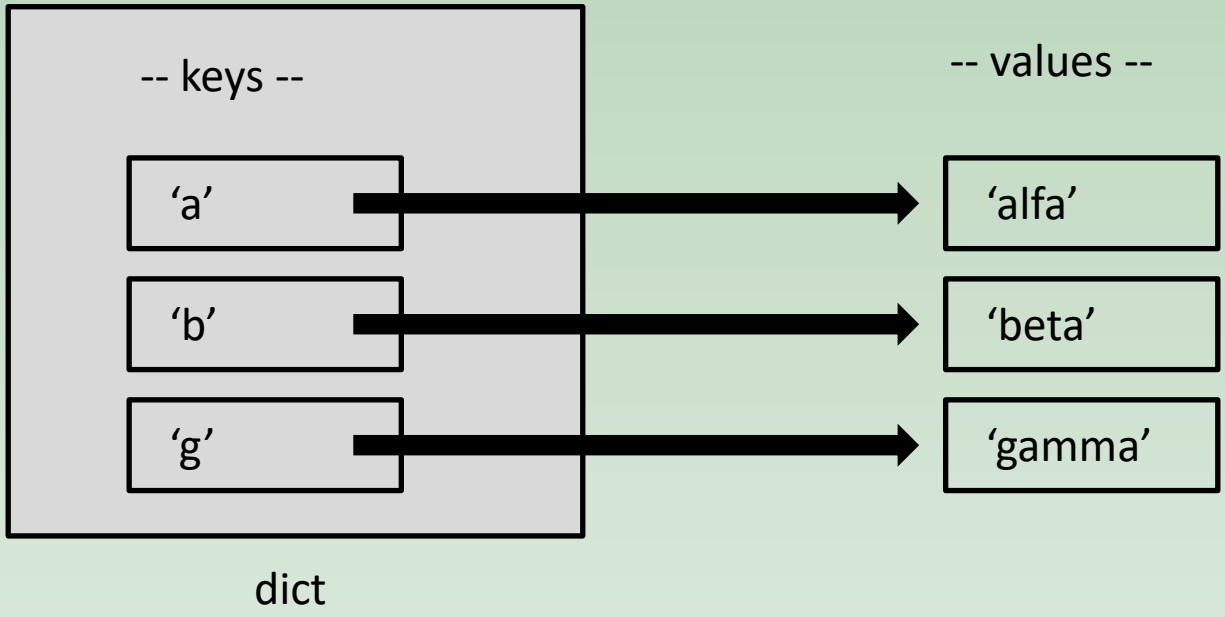
dictionary

for storing **key / value** pairs

```
>>> d = { }
>>> d['a'] = 'alfa'
>>> d['b'] = 'beta'
>>> d['g'] = 'gamma'
>>> d
{'a': 'alfa', 'b': 'beta', 'g': 'gamma'}
>>> d['a']
'alfa'
>>> d['o']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'o'
>>> d.get('o')
>>> d.get('a')
'alfa'
>>> 'a' in d
True
>>> 'o' in d
False
>>> d['a'] = 'ALFA'
>>> d['a']
'ALFA'
>>> d
{'a': 'ALFA', 'b': 'beta', 'g': 'gamma'}
```

empty dictionary
(or: `d = dict()`)

Is an element with the given key in the dictionary?


-- keys --
-- values --
‘a’
‘b’
‘g’
‘alfa’
‘beta’
‘gamma’
dict

```
>>> d
{'b': 'beta', 'g': 'gamma', 'a': 'alfa'}
>>> d['o'] = 'omega'
>>> d
{'o': 'omega', 'b': 'beta', 'g': 'gamma', 'a': 'alfa'}
>>> d.keys()
dict_keys(['o', 'b', 'g', 'a'])
>>> type(d.keys())
<class 'dict_keys'>
>>> d.values()
dict_values(['omega', 'beta', 'gamma', 'alfa'])
>>> type(d.values())
<class 'dict_values'>
>>> for k in sorted(d.keys()):
...     print("key:", k, "=>", d[k])
...
...
key: a => alfa
key: b => beta
key: g => gamma
key: o => omega
>>> d.items()
dict_items([('o', 'omega'), ('b', 'beta'), ('g', 'gamma'), ('a', 'alfa')])
>>> type(d.items())
<class 'dict_items'>
>>> for k, v in d.items():
...     print(k, v)
...
o omega
b beta
g gamma
a alfa
>>> list(d.keys())
['o', 'b', 'g', 'a']
```

the order of the elements reflects the order of insertion since Python 3.7

iterators:
d.keys()
d.values()
d.items()

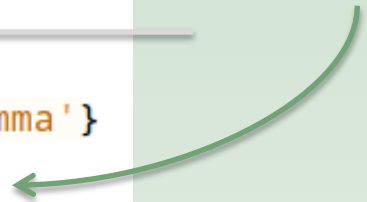
using iterators in a loop

Iterator to list.
The order is arbitrary.

HW: dict1.py

removing an element from a dictionary

```
>>> a = 6
>>> a
6
>>> del a
>>> a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a' is not defined
>>>
>>> li = range(5)
>>> li
[0, 1, 2, 3, 4]
>>> del li[-1]
>>> li
[0, 1, 2, 3]
>>>
>>> d
{'a': 'alfa', 'b': 'beta', 'o': 'omega', 'g': 'gamma'}
>>> del d['b']
>>> d
{'a': 'alfa', 'o': 'omega', 'g': 'gamma'}
```



Exercises

homework

1. [20120904a] removing duplicates (set)
2. [20120905a] dictionary #1
3. [20120921a] accent removal
4. [20130218b] certain characters
5. [20120818h] one hundred 50-digit long numbers (PE #13) [version **A** only]
6. [20120816a] 8 queens